

Arquivos

Uma das importantes práticas de programação para profissionais e estudantes que desenvolvem serviços e aplicações é a devida manipulação de arquivos em seus diferentes tipos e propósitos de armazenamento. Mesmo sendo um formato pioneiro no transporte e armazenamento em memória secundária – termo utilizado para se referir a memórias não voláteis, como discos rígidos e memórias de estado sólido –, os arquivos são amplamente utilizados para finalidades como intercâmbio de dados entre aplicações (JSON, XML, CSV etc.) e configurações de serviços (YAML, INI, ENV etc.), sem esquecer dos arquivos com dados fora de um padrão específico, conhecidos como dados RAW (Guilhon *et al.*, 2022; Lambert, 2022).

Além da família de arquivos texto, existem também os arquivos binários com formatos conhecidos, tais como mídias nos seus diferentes tipos (JPEG, PNG, MP3, MP4, AVI etc.) e arquivos com dados compactados ou executáveis (ZIP, ARJ, EXE, BIN etc.), cada um com suas particularidades na manipulação.

Este capítulo tem por propósito apresentar as práticas de programação voltada para a manipulação dos diferentes tipos de arquivos, destacando suas particularidades e finalidades de uso. O primeiro subcapítulo apresenta como arquivos textos são importantes no intercâmbio de dados, incluindo suas técnicas de manipulação. Depois, serão mostradas as particularidades na manipulação de arquivos binários.

1 Arquivos texto

Na programação em linguagem Python, a manipulação de textos é nativa, contendo funções e objetos que facilitam sua manipulação. Existem algumas particularidades quando se deseja trabalhar com arquivos textos que precisam ser levados em conta, tal como a codificação, de forma que cada solução de software adote um conjunto de caracteres dentro de um padrão, o qual está correlacionado com o espaço que esse caractere ocupa em memória secundária, além de questões regionais em sua adoção. O quadro 1 apresenta as principais codificações de arquivos existentes. Entre as codificações mais populares, encontra-se a UTF-8 (padrão utilizado em idiomas do Ocidente) ou UTF-16 (padrão de 2 ou 4 bytes utilizado em sistemas asiáticos).

Quadro 1 – Codificações de arquivos

CODIFICAÇÃO	CARACTERÍSTICAS PRINCIPAIS	USO COMUM
UTF-8	Codificação universal, compatível com ASCII, suporta todos os caracteres Unicode	Padrão na web, Python, JSON, arquivos modernos
UTF-16	Representa caracteres Unicode com 2 ou 4 bytes, pode conter byte order mark (BOM)	Windows, arquivos XML, sistemas asiáticos
UTF-32	Usa 4 bytes fixos por caractere, muito simples, mas ocupa mais espaço	Casos específicos que exigem performance previsível
ASCII	Codifica apenas 128 caracteres (inglês básico), leve e compatível com sistemas antigos	Arquivos simples, sistemas legados

cont.

CODIFICAÇÃO	CARACTERÍSTICAS PRINCIPAIS	USO COMUM
ISO-8859-1	Também conhecida como Latin-1, inclui acentos usados em português e outras línguas latinas	Sistemas legados e antigos arquivos em português
Windows-1252	Extensão do ISO-8859-1 com símbolos extras	Windows antigos, arquivos Word etc.



PARA SABER MAIS

Ao lidar com arquivos de texto em Python (e em praticamente qualquer linguagem de programação), é comum nos depararmos com termos como UTF-8, UTF-16 ou Unicode. Mas, afinal, o que significa Unicode?

Unicode é um padrão universal de codificação de caracteres, criado para resolver um problema antigo: a falta de um sistema único e consistente para representar todos os símbolos usados nas linguagens humanas – incluindo letras, números, acentos, emojis, alfabetos não latinos e símbolos técnicos.

Antes do Unicode, cada sistema ou país criava seu próprio conjunto de códigos, como o ASCII, ISO-8859-1 ou Windows-1252, o que gerava confusões e arquivos incompatíveis entre si. O Unicode surgiu para unificar tudo isso, atribuindo a cada caractere um código numérico único, chamado de code point.

Depois de reconhecer os diferentes tipos de codificação existentes, a próxima etapa é entender o processo primordial de arquivos, isto é, sua abertura e fechamento. A abertura consiste em receber um objeto no qual se faz a gestão de manipulação de arquivos: localiza-se o arquivo, disponibiliza-se para manipulação e depois realiza-se o fechamento.

O Python possui duas estruturas nativas na manipulação de arquivos em que a diferença está no processo de fechamento do arquivo. O primeiro exemplo está relacionado ao uso da função `open()`, a qual necessariamente abre um arquivo, imprime o conteúdo e faz o fechamento, como segue.

Exemplo 1

```
arquivo = open("exemplo.txt", "r") # abre para leitura
conteudo = arquivo.read()
print(conteudo)
arquivo.close()
```

Algumas considerações são importantes no código-fonte apresentado no exemplo 1. Primeiramente, no comando `open()`, é passado como argumento o caminho do arquivo (`exemplo.txt`) para a uma operação de leitura (destacado com `"r"`). Na abertura do arquivo, podem ser escolhidos diferentes tipos de operação, permitindo a escrita, leitura ou ambas as operações. O quadro 2 mostra os possíveis modos de abertura do arquivo.

Quadro 2 – Modos de abertura

MODO	SIGNIFICADO	DESCRIÇÃO
'r'	Read	Abre para leitura. Gera erro se o arquivo não existir.
'w'	Write	Abre para escrita. Cria o arquivo se não existir, sobrescreve se existir.
'a'	Append	Abre para adição no final do arquivo. Cria o arquivo se não existir.
'x'	Create	Cria um arquivo. Gera erro se o arquivo já existir.
'b'	Binary	Modo binário (adicionado a outros modos, como 'rb' ou 'wb').
't'	Text (padrão)	Modo texto (padrão, pode ser omitido). Usado com 'r', 'w' etc.
'+'	Read and write	Abre para leitura e escrita ao mesmo tempo. Usado com 'r+', 'w+' etc.



IMPORTANTE

Ao lidar com Python, os modos de abertura de arquivos podem ser combinados para oferecer mais controle sobre como o arquivo será manipulado. Por exemplo, ao utilizar `'rb'`, o arquivo é aberto para leitura em modo binário – ideal para imagens, áudios e vídeos, que não são lidos como

texto. Já o modo 'w+' permite escrever e ler no mesmo arquivo, mas apaga todo o conteúdo anterior caso o arquivo já exista. O modo 'a+' é útil quando se deseja adicionar novos dados ao final de um arquivo existente, sem apagar o conteúdo existente, ao mesmo tempo que se mantém a possibilidade de leitura. Outro exemplo é 'x+', que permite criar um arquivo para leitura e escrita, gerando erro se o arquivo já existir, o que o torna útil em situações em que é importante evitar sobrescritas acidentais. Esses modos combinados dão mais flexibilidade ao programador, mas exigem atenção para evitar perda de dados ou comportamentos inesperados.

O próximo exemplo é realizar a abertura do arquivo, porém, utilizando a estrutura `with open`, a qual tem como particularidade a garantia do fechamento do arquivo. A estrutura estabelece que toda a operação com o arquivo deve ser realizada dentro de um bloco de execução. Quando o programa encerra a execução do bloco, automaticamente o arquivo é fechado. Vamos compreender com o exemplo 2.

Exemplo 2

```
with open("exemplo.txt", "r") as arquivo:
    conteudo = arquivo.read()
    print(conteudo)
# Arquivo é fechado automaticamente aqui
```

Perceba que toda a manipulação do arquivo é realizada dentro do bloco estabelecido. Quando encerrada a execução do bloco, o arquivo é encerrado e o objeto *arquivo* é desalocado da memória da aplicação, evitando esquecimentos no encerramento de arquivos.

Também existem outras maneiras de realizar a leitura do arquivo utilizando comandos como o `readline()`, que faz a leitura completa de uma linha por execução, e o comando `readlines()`, que faz a leitura de todo o arquivo organizado em listas de linhas, como demonstra o exemplo 3.

Exemplo 3

```
with open("exemplo.txt", "r") as arquivo:
    linha1 = arquivo.readline()
    linhas = arquivo.readlines()
    print(linha1)
    print(linhas)
```

Para a escrita em arquivos, são utilizados comandos como o `write()`, em que se escreve um texto qualquer no arquivo, e o comando `writelines()`, em que se pode escrever um conjunto de linhas organizado por listas. Um cuidado importante na escrita é a escolha do modo de abertura: "w" indica que o arquivo será sobrescrito, e "a" indica que o conteúdo será gravado ao final do arquivo.

Exemplo 3

```
# Sobrescreve o conteúdo
with open("saida.txt", "w") as arquivo:
    arquivo.write("Primeira linha.\n")
    arquivo.write("Segunda linha.\n")

# Adiciona ao final do arquivo
with open("saida.txt", "a") as arquivo:
    arquivo.write("Linha adicionada.\n")
```

No exemplo 3, identificamos que, na primeira manipulação do arquivo `saida.txt`, o segundo comando `write()` sobrescreverá a escrita anterior, enquanto a segunda manipulação está com o modo de abertura "a" e sempre adicionará uma nova linha ao final.

Outros dois pontos importantes na manipulação de arquivos estão na escolha da codificação e no tratamento de erros. Antes de realizar a abertura de um arquivo, pode ser especificado o argumento `encoding` para garantir que tal manipulação respeita um tipo de codificação. Veja um exemplo em que é informado que o padrão da codificação é UTF-8. Essa configuração evita problemas com acentos e caracteres especiais.

Exemplo 4

```
with open("exemplo.txt", "r", encoding="utf-8") as arquivo:
    print(arquivo.read())
```

Para evitar encerramentos desnecessários na aplicação, as funções e os objetos de manipulação de arquivos geram exceções para algumas situações; por exemplo, quando o arquivo não é encontrado ou quando a aplicação não possui autorização para acesso. Para capturar e tratar a exceção, o código-fonte deve ser circundado com o bloco try, e cada exceção pode ter sua resposta programada de maneira personalizada utilizando bloco except, como mostra o exemplo 5.

Exemplo 5

```
try:
    with open("inexistente.txt", "r") as arquivo:
        conteudo = arquivo.read()
except FileNotFoundError:
    print("Arquivo não encontrado.")
```

No exemplo 5, o programa está preparado para uma situação caso o arquivo não seja encontrado, fornecendo uma resposta amigável. O quadro 3 apresenta as principais exceções que podem ser capturadas no processo de manipulação de arquivos.

Quadro 3 – Exceções para arquivos em Python

EXCEÇÃO	QUANDO OCORRE	EXEMPLO DE SITUAÇÃO
FileNotFoundError	Quando o arquivo não é encontrado no modo de leitura ('r' ou 'r+')	Tentar abrir um arquivo que não existe
PermissionError	Quando o programa não tem permissão para acessar ou modificar o arquivo	Tentar abrir um arquivo de sistema sem privilégios
IsADirectoryError	Quando se tenta abrir uma pasta como se fosse um arquivo	Usar open("caminho/") em vez de open("arquivo.txt")

cont.

EXCEÇÃO	QUANDO OCORRE	EXEMPLO DE SITUAÇÃO
NotADirectoryError	Quando se tenta acessar um caminho inválido, tratando um arquivo como pasta	<code>open("arquivo.txt/outro.txt")</code>
IOError ou OSError	Erros genéricos de entrada/saída (I/O), como falhas de leitura, disco cheio etc.	Problema físico ou lógico ao acessar o arquivo
UnicodeDecodeError	Quando a leitura falha por usar a codificação errada	Tentar ler um arquivo em UTF-16 usando <code>encoding='utf-8'</code>
UnicodeEncodeError	Quando ocorre erro ao tentar salvar texto com caracteres incompatíveis	Tentar salvar acentos em um arquivo com <code>encoding='ascii'</code>
ValueError	Quando o modo de abertura do arquivo é inválido ou está mal formatado	Usar <code>open("arquivo.txt", "rw")</code> , que não é um modo válido

2 Arquivos binário

Diferentemente da manipulação de dados em texto, os arquivos binários, como o nome sugere, possuem uma manipulação refinada a nível de cada byte, levando em consideração as diferentes codificações particulares, principalmente voltadas para arquivos executáveis, compactados ou de mídia.

Os arquivos, quando sinalizados para serem tratados com formatos binários, não passam por decodificação automática tal como os arquivos textos UTF-8 ou ASCII. Os arquivos, dessa maneira, são disponibilizados em formato de um conjunto de bytes com formato bytearray. O exemplo 6 apresenta o uso do comando `with open` na abertura de um arquivo binário.

Exemplo 6

```
with open("audio.mp3", "rb") as arquivo:
    dados = arquivo.read(1024) # lê os primeiros 1024 bytes
```

Nesse caso, é importante observar que a sinalização para a manipulação do arquivo em formato binário é a inclusão da letra “b” no modo de abertura; nesse caso, “rb”, em que o arquivo será aberto em formato binário para leitura *sem a decodificação automática*. O comando `read()`, por sua vez, armazena na variável `dados` os primeiros 1024 bytes do arquivo `audio.mp3`.

Com o uso de laços, é possível determinar os blocos para processamento de todo um arquivo binário. No exemplo a seguir, um arquivo de vídeo MP4 pode ser processado em divisões de 4KB utilizando o comando `read()` em conjunto com o laço `while` até que a leitura alcance o final do arquivo (chamado de EOF – end of file). O operador `:=`, conhecido como “operador walrus” (ou “operador morsa”), foi introduzido no Python 3.8 e permite a atribuição de valores a variáveis dentro de expressões. Ele é especialmente útil para evitar repetições, simplificar códigos e torná-los mais concisos, como em estruturas condicionais ou loops. Por exemplo, em vez de escrever um bloco de código em que uma função é chamada duas vezes (uma para verificar o valor retornado e outra para usá-lo) o operador `:=` permite que o resultado seja atribuído a uma variável e usado simultaneamente na expressão. Seu uso comum é dentro de um `while`, como em `while (linha := arquivo.readline())`; que lê uma linha do arquivo e avalia se ainda há conteúdo; tudo em uma única instrução. Essa funcionalidade favorece a legibilidade e eficiência do código, desde que usada com parcimônia para não prejudicar a clareza.

Exemplo 7

```
with open("video.mp4", "rb") as arquivo:
    while chunk := arquivo.read(4096):
        # processa bloco de 4KB
        pass
```

Um outro exemplo de operação com arquivos binários é realizar sua cópia combinando os comandos `write()` e `read()` de maneira indentada, como segue.

Exemplo 8

```
with open("original.pdf", "rb") as origem:
    with open("copia.pdf", "wb") as destino:
        destino.write(origem.read())
```



IMPORTANTE

Os arquivos binários são fundamentais para o funcionamento de diversas aplicações no mundo digital, pois permitem o armazenamento e a manipulação eficiente de dados complexos que não podem ser representados apenas como texto, imagens, vídeos, áudios, documentos, planilhas e programas executáveis. Diferentemente dos arquivos de texto, que armazenam caracteres legíveis, os arquivos binários contêm sequências de bytes que representam diretamente estruturas internas, compactadas ou codificadas, exigindo programas específicos para sua interpretação. Essa capacidade de representar dados ricos em informação, com formatos compactos e performáticos, torna os arquivos binários indispensáveis em áreas como multimídia, jogos, sistemas embarcados, bancos de dados, engenharia e até mesmo no armazenamento de modelos de inteligência artificial. Em Python, trabalhar com arquivos binários exige atenção especial ao modo de abertura ('rb', 'wb') e à manipulação de dados no formato de bytes, garantindo precisão e fidelidade na leitura e gravação dessas informações.

Considerações finais

A manipulação de arquivos é uma competência essencial na programação, pois permite que aplicações armazenem, leiam e troquem informações de forma eficiente e persistente. Ao longo deste capítulo, foram apresentadas as principais práticas relacionadas à manipulação de arquivos textos e binários na linguagem Python, destacando particularidades, modos de abertura, cuidados com codificação, tratamento de exceções e formas de leitura e escrita. Os arquivos textos, amplamente utilizados para configurações e intercâmbio de dados entre sistemas, exigem

atenção à codificação de caracteres, enquanto os arquivos binários são fundamentais para lidar com dados complexos, como imagens, áudios e documentos. Compreender esses conceitos permite que o programador atue de forma mais segura, organizada e eficiente no desenvolvimento de soluções que dependem de leitura e gravação em memória secundária, respeitando os formatos e padrões esperados em cada tipo de aplicação.

Referências

GUILHON, André *et al.* (org.). **Jornada Python**: uma jornada imersiva na aplicabilidade de uma das mais poderosas linguagens de programação do mundo. Rio de Janeiro, RJ: Brasport, 2022. *E-book*.

LAMBERT, Kenneth A. **Fundamentos de Python**: estruturas de dados. Cengage Learning Brasil, 2022. *E-book*.

